Sri Lanka Institute of Information Technology

IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 07

Student ID	IT24101365
Student Name	Kamshiga Ganesan
Campus/ Center name	Northern UNI
Specialization	Software Engineering
Batch No	1

Question 1

```
kamshiga@kamshiga-VirtualBox: ~                    kamshiga@kamshig
 GNU nano 7.2                                         k5.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void function_A(){
 printf("Child process executing\n");
}
int main(){
 int i;
 pid_t pid;
 for(i=0; i<10; i++)
 {
  if((pid=fork()) <0){
    printf("Fork Failed\n"); //error
  }
  else if(pid ==0)
  {
   function_A();
   return 0;
  }
  printf("process ID: %d\n",pid);//Line A
 }
 for(i=0; i<10; i++)//Line B
   wait(NULL);
 return 0;
```

```
kamshiga@kamshiga-VirtualBox:~$ nano k5.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o k5 k5.c
kamshiga@kamshiga-VirtualBox:~$ ./k5
process ID: 6172
Child process executing
Child process executing
process ID: 6173
process ID: 6174
process ID: 6175
process ID: 6176
Child process executing
Child process executing
Child process executing
Child process executing
process ID: 6177
process ID: 6178
process ID: 6179
Child process executing
Child process executing
process ID: 6180
Child process executing
process ID: 6181
Child process executing
kamshiga@kamshiga-VirtualBox:~$
```

i)How many new processes are created in the program? Why?

-10 new processes are created.

Because the loop runs 10 times and in each iteration fork() creates one new child process.

ii) Which process, the parent or the child, executes function_A()? Why?

-Child process executes functionA()

Because if(pid==0), this condition is true only for the child process.

iii) Whose PID, the parent or the child, is printed in Line A? Why?

- Child's PID is printed by the parent process

Because:

- In parent - pid contains child PID
- In child - pid = 0, but child returns before printing

iv) What is the purpose of the loop (with its wait()) in Line B?

-To wait for all child processes to finish and avoid zombie processes.

Question 2

i)For N=5, How many processes are created when the program is executed?

-Total processes = 2^5 = 32 processes

ii) Modify the program so that only the parent process creates 3 child processes, and each new created process calls a function CPU(). In addition, make the parent process wait for each child's termination.

```
  GNU nano 7.2                                                k1.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void CPU(){
 printf("Child process running CPU function\n");
}

int main(){
 int i;
 pid_t pid;

 for(i=0; i<3; i++){
  pid=fork();

  if(pid==0){
   CPU();
   return 0;
  }
 }

 return 0;
}
```

```
Mar 20  14:28
kamshiga@kamshiga-VirtualBox: ~
kamshiga@kamshiga-VirtualBox:~$ nano k1.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o k1 k1.c
kamshiga@kamshiga-VirtualBox:~$ ./k1
Child process running CPU function
Child process running CPU function
Child process running CPU function
kamshiga@kamshiga-VirtualBox:~$ 
```

Question 3

Consider the following program. What will be the output in Line A?

-PARENT: value =40

```
  GNU nano 7.2                                            k2.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int value=60;

int main(){
 pid_t pid;
 pid=fork();
 if(pid==0){
  value=value+20;
 }
 else if(pid>0){
  value=value-20;
  printf("PARENT: value=%d\n", value);
  wait(NULL);
 }
 return 0;
}

                                                    [ Read 21 lines ]
```

```
kamshiga@kamshiga-VirtualBox:~$ nano k2.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o k2 k2.c
kamshiga@kamshiga-VirtualBox:~$ ./k2
PARENT: value=40
kamshiga@kamshiga-VirtualBox:~$
```

Question 4

Consider the following programs A and B.

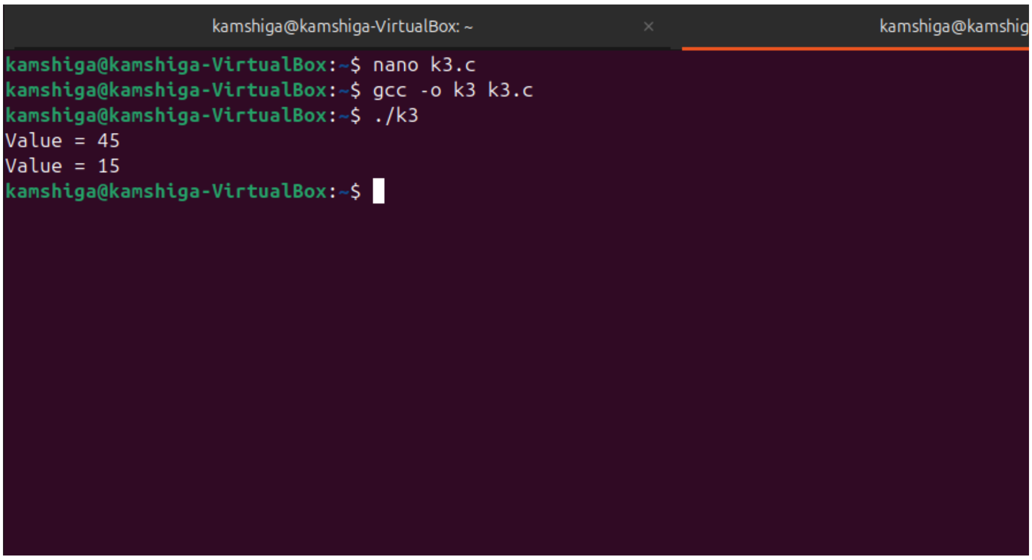
i)How many times will the fork () function be called in Program A? (i.e., how many processes are created?) Justify your answer.

-Total processes = 2^4 = 16 processes

-Each fork() create a new process. Every process continues the loop and calls fork() again.So that number of processes doubles in each iteration.

ii)What is the output of Line A in Program B? Justify your answer.

```
  GNU nano 7.2                                            k3.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int value =30;

int main() {
 pid_t pid;
 pid= fork();

 if(pid == 0){
  value =value+15;
 }
 else if(pid >0){
  value =value-15;
  wait(NULL);
 }

 printf("Value = %d\n", value);    //Line A

 return 0;
}
```



-When the fork() system call is executed, it creates a child process. After the fork, both the parent and the child processes have separate copies of the variable value. In the child process (where pid == 0), the value is increased by 15, making it 45. In the parent process (where pid > 0), the value is decreased by 15, making it 15. Both the parent and child processes then execute the printf() statement, which results in two outputs being printed. The order of the output may vary because the parent and child processes execute concurrently.

Question 5

```
  GNU nano 7.2                                        k4.c
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 25

void* thread_routine (void* x)
 {
  printf("I'm Thread %2ld my TID is %lu\n" ,(long) x,(unsigned long) pthread_self() );
  pthread_exit(0);
 }

int main() {
 pthread_attr_t thread_attr;
 pthread_t tids[NUM_THREADS];
 int x;

 pthread_attr_init (&thread_attr);
 for(x=0; x< NUM_THREADS; x++)
 {
  pthread_create(&tids[x], &thread_attr, thread_routine,(void*)(long)x );
 }

 printf("Waiting for threads to finish\n");
```

```
 for(x=0; x<NUM_THREADS; x++)
 {
  pthread_join (tids[x],NULL);
 }

 printf("All threads are now finshed\n");

 return 0;
}
```

```
kamshiga@kamshiga-VirtualBox:~$ nano k4.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o k4 k4.c
kamshiga@kamshiga-VirtualBox:~$ ./k4
I'm Thread  1 my TID is 132759997638336
I'm Thread  0 my TID is 132760006031040
I'm Thread  9 my TID is 132759855036096
I'm Thread  3 my TID is 132759980852928
I'm Thread  2 my TID is 132759989245632
I'm Thread  4 my TID is 132759972460224
I'm Thread  6 my TID is 132759955674816
I'm Thread  8 my TID is 132759938889408
I'm Thread  5 my TID is 132759964067520
I'm Thread  7 my TID is 132759947282112
I'm Thread 10 my TID is 132759846643392
I'm Thread 11 my TID is 132759838250688
I'm Thread 12 my TID is 132759829857984
I'm Thread 13 my TID is 132759821465280
I'm Thread 14 my TID is 132759813072576
I'm Thread 15 my TID is 132759804679872
I'm Thread 16 my TID is 132759796287168
I'm Thread 17 my TID is 132759787894464
I'm Thread 18 my TID is 132759779501760
I'm Thread 19 my TID is 132759771109056
I'm Thread 20 my TID is 132759762716352
I'm Thread 22 my TID is 132759745930944
Waiting for threads to finish
I'm Thread 23 my TID is 132759737538240
I'm Thread 21 my TID is 132759754323648
I'm Thread 24 my TID is 132759729145536
All threads are now finshed
kamshiga@kamshiga-VirtualBox:~$
```

1. What is the role of pthread_t tids[NUM_THREADS]?

- Stores thread IDs of all created threads.

2. What is the purpose of the for loop that calls pthread_create()?

- To create 25 threads, each running thread_routine()

3. What is the role of the function thread_routine()?

- Function executed by each thread - prints thread number and ID

4. Why is pthread_join() used in the second for loop?

- To wait for all threads to finish execution before program ends